Sentiment Classification of IMDB Movie Reviews Using Classical NLP Techniques

Group G07: Sadiya Mohon Auti, Tasnim Tabassum Mim
Department of Computer Science
American International University–Bangladesh, Dhaka, Bangladesh
IDs: 23-52092-2, 23-50971-1
Colab notebook (view access): Click Here

***Abstract*—We compare and develop classical natural language processing (NLP) models that categorize positive and negative movie reviews in English. We use a stratified sample of 10,000 reviews from the IMDB Large Movie Review Dataset after deleting the 418 duplicate reviews, and split this sample into 70% training, 15% validation and 15% test data. We analyze how different preprocessing methods (removing stop-words, keeping or ignoring negations, and stemming versus lemmatisation) affect our results, and we compare Bag-of-Words, TF-IDF (unigram and unigram plus bigram) and averaged Word2Vec representations. The results are compared using two machine-learning algorithms, Multinomial Naive Bayes and Logistic Regression, with all hyperparameters tuned on the validation set. The best model, TF-IDF (unigram plus bigram) with Logistic Regression ($C = 10$), achieved a test macro-F1 score of 0.8847, clearly better than the majority-class baseline (0.3333). An error analysis demonstrated that most errors were caused by negation (14 out of 20 examples studied), sarcasm and mixed opinions.**

***Index Terms*—sentiment analysis, text classification, TF-IDF, Naive Bayes, Word2Vec, IMDB**

I. Introduction

The opinions expressed in online movie reviews affect what movies consumers choose to view. Streaming services and review websites therefore want to automatically summarize audience opinions on movies, rank them and find reviews in which there is a discrepancy between the reviewer's rating (positive or negative) and the content of the review text, without having to manually read through thousands of individual reviews. At its core this is binary sentiment classification: given the text of a review, predict whether it is positive or negative.

The problem of classifying sentiment is more difficult than that of identifying topics from text, since the same words used to express a particular idea may have completely different interpretations depending on their context: "good" and "not good", although they represent opposing ideas, are separated by only a single word ("not") that is treated as a common stop-word. These differences make the choice of preprocessing methods critical, and thus motivate us to evaluate multiple pipelines in a controlled study instead of fixing a single pipeline in advance.

The main contributions of this paper are: (1) a flexible preprocessing method that allows users to toggle between three primary options: stop-word removal, negation handling, and stemming/lemmatisation; (2) an experiment that provides an ablation study of these three primary preprocessing options; (3) an experiment comparing four text representation methods combined with two classification methods, using the exact same evaluation criteria across both experiments, where all results were evaluated using a test set that was withheld during training; and (4) a final experiment providing an error analysis of our top-performing model. We do not utilize any advanced techniques such as transformer-based architectures and focus instead solely upon classical approaches.

II. Related Work

Pang *et al.* [1] were some of the first people who treated sentiment as a classification task for movie reviews. In this research they tested three different methods using Naive Bayes, Maximum Entropy and Support Vector Machines (SVM) for their effectiveness in classifying sentiments. These methods all produced good results but were much worse at identifying sentiment than methods used to classify topics. Maas *et al.* [2] created the IMDB Large Movie Review Dataset with 25,000 training and 25,000 testing entries and then demonstrated that building word vectors that capture the aspects of words bearing sentiment produces better results than the standard Bag-of-Words model. Wang and Manning [3] also demonstrated that simple baseline approaches can be effective; particularly when it comes to tasks involving sentiment, bigram features produce consistent improvements and Naive Bayes was competitive, even when working with very short documents. This is why we chose to use unigram + bigram TF-IDF and Naive Bayes in our approach. Mikolov *et al.* [4] developed Word2Vec (a method for training word embeddings efficiently). We applied this method to develop document vectors by taking the average of each vector.

III. Dataset and EDA

We used the IMDB Large Movie Review Dataset [2] made available to researchers by Maas *et al.*, which we downloaded using the public Hugging Face mirror *stanfordnlp/imdb*. It has 50,000 labeled movie reviews (25,000 train, 25,000 test) and was perfectly balanced when first released. Since the reviews were originally split into two separate groups, we combined them all together and removed any duplicate reviews that existed across the entire collection (there were 418 duplicates out of the 50,000 total reviews; after removing these, 49,582

unique reviews remained, which prevented the same review from appearing in both an original test and train split). Then, using a fixed seed value, we created a stratified random sample of 10,000 reviews (5,000 positive, 5,000 negative) and randomly assigned them into three sets (training = 7,000 reviews; validation = 1,500 reviews; test = 1,500 reviews).

Class Distribution. As mentioned above, our training dataset contains exactly 3,500 positive and 3,500 negative reviews. The dataset is therefore perfectly balanced (Fig. 1). In addition to accuracy, we use macro-F1 as our primary evaluation metric since macro-F1 treats both classes equally.

Word Length. Our reviews have an average of approximately 231 words per review and a median of about 173 (with one review being nearly 2,300 words long). The length of the reviews follows a highly skewed distribution where many reviews are short but some are much longer than most others (Fig. 2). The two classes have almost the same length distribution (mean 229.7 vs. 233.0 words), so length alone is not a useful feature.

Frequent Words. The list of the top 15 words before removal of HTML tags and stop words includes twelve words that are common to both classes. These include common English stop words such as "the", "a", "and" and HTML tag fragments such as "br" (Fig. 4, top row). After removal of stop-words and HTML tags, the sentiment-specific vocabulary is more easily apparent in each of the two classes. With a lighter cleaning step (Fig. 3), ten of the 15 most frequent words are shared by both classes; class-specific words are "bad", "could", "get", "make", "people" (negative) and "also", "first", "great", "love", "well" (positive). For example, as shown in Fig. 3, the top words in the negative class are primarily related to expressing dislike or dissatisfaction ("bad", "worst", etc.). Raw frequency therefore separates the classes only weakly, and a weighted model is needed.

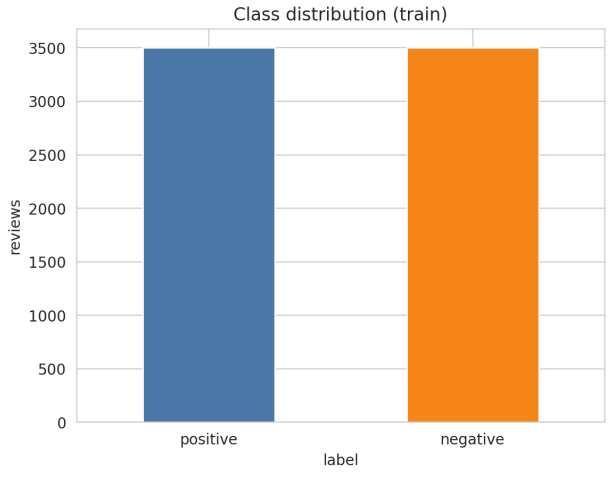


Fig. 1. Class distribution of the training split: perfectly balanced.

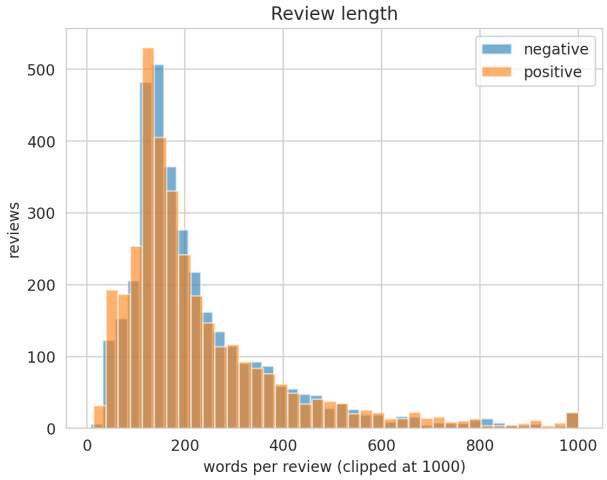


Fig. 2. Review length by class: right-skewed, nearly identical for both classes.

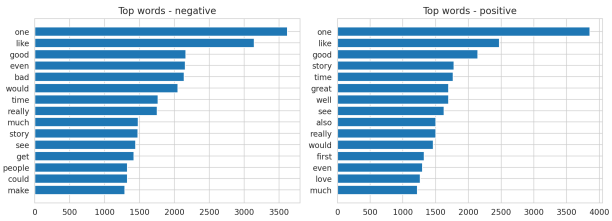


Fig. 3. Top-15 words per class after light cleaning and stop-word removal ("movie" and "film" excluded): most are shared by both classes.

IV. Methodology

A. Preprocessing pipeline

The pipeline is a single Python function (Fig. 5) whose switches (stop-word removal, negation kept or removed, stemming, lemmatisation or none) drive the ablation. The steps are: raw review → lower-casing → regex cleaning (HTML tags, URLs, digits) → contraction expansion (*don't* → *do not*) → alphabetic tokenisation → stop-word removal → stemming or lemmatisation. Each step has a reason:

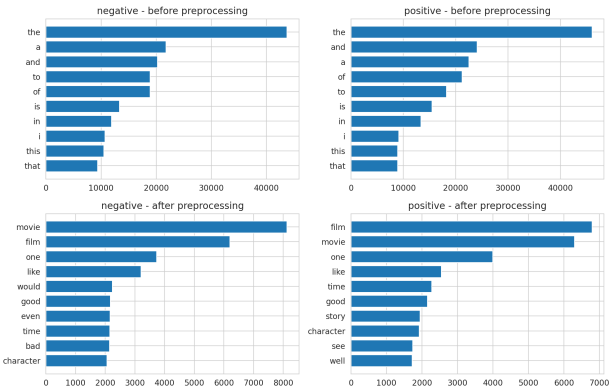


Fig. 4. Top-10 words per class before (top) and after (bottom) preprocessing: function words disappear.

- **Regex cleaning** strips out `<br />` tags, URLs and digits.
- **Contraction expansion** keeps negation visible ("not" survives as a separate token).
- **Stop-word removal** uses the NLTK English list and shrinks the vocabulary; the list contains negations (*not, no, nor*), which can be kept (Variant C) or removed (Variant B).
- **Normalisation** uses the Porter stemmer or the WordNet lemmatiser.

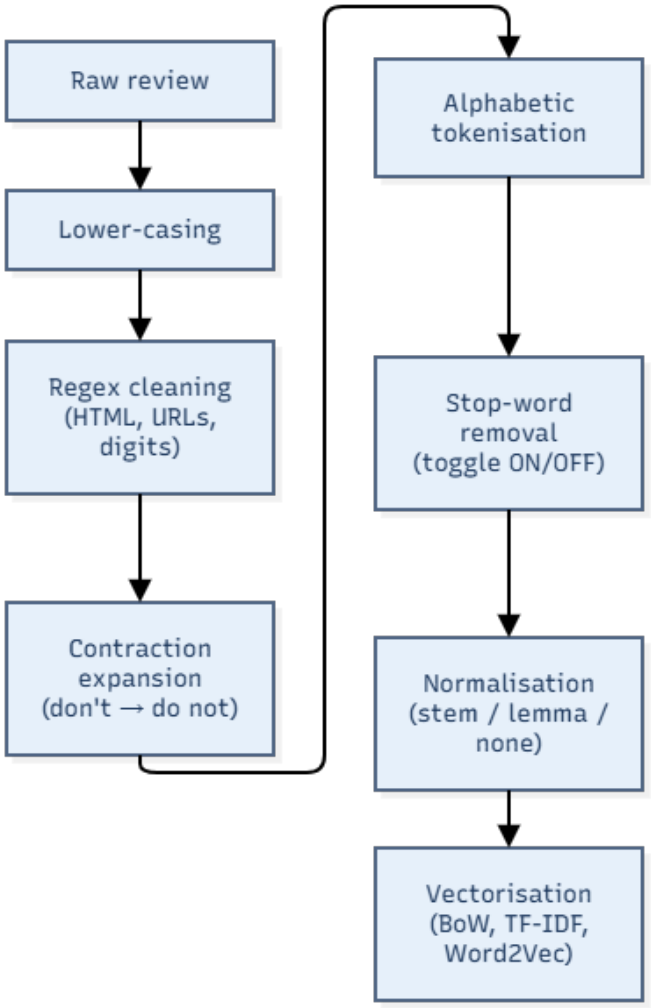


Fig. 5. Preprocessing pipeline: raw review → lower-casing → regex cleaning → contraction expansion → tokenisation → stop-word removal (switch) → normalisation (switch) → vectorisation.

B. Representations

We compare four feature sets, all fitted on the training split only:

1) **Bag of Words**: counts of the 20,000 most frequent words (min_df = 3).
2) **TF-IDF unigram**: 20,000 features, L2-normalised, which damps the effect of very long reviews.
3) **TF-IDF unigram + bigram**: 50,000 features; bigrams such as "not good" capture short phrases that a single word cannot.
4) **Averaged Word2Vec**: a 100-dimensional model trained on the training reviews only, representing each review by the mean of its word vectors.

We expected TF-IDF with bigrams to work best because reviews are long enough for sparse features to be estimated reliably and because phrase information helps; the dense averaged embeddings are included to test whether a compact representation can compete.

C. Classifiers and tuning

We use a majority-class baseline, **Multinomial Naive Bayes** (BoW and TF-IDF only) and **Logistic Regression** (all representations). Naive Bayes is a fast, strong text baseline and gives interpretable word weights for the error analysis; Logistic Regression learns discriminative weights and works with both sparse and dense inputs. We tuned Laplace smoothing $\alpha \in \{0.1, 0.3, 1, 3\}$ for NB and regularisation strength $C \in \{0.1, 1, 10, 100\}$ for LR on the validation set by macro-F1.

V. Experimental Setup

The dataset was divided in the ratio 70/15/15 (with seed = 42). Each model was trained using the training subset, with hyperparameter selection made on the validation subset. The test subset was then used for the final evaluation. Accuracy, macro-precision, macro-recall and macro-F1 were evaluated, with macro-F1 as the primary measure. Implementation: Python 3.13.18, scikit-learn 1.6.1 [5], gensim 4.4.0, NLTK 3.9.1, pandas 2.2.3, numpy 2.4.6 (versions printed by the submitted notebook [6]).

VI. Results

A. Preprocessing ablation

Table I compares five pipeline variants with TF-IDF unigrams and Logistic Regression ($C = 1$) on the validation set. Interestingly, Variant B (removing stop-words including negation) achieved the highest validation macro-F1 (0.8899), slightly outperforming Variant C (0.8866) which kept the negation. This suggests that, for this TF-IDF model, removing all stop-words reduced the noise enough to compensate for the loss of the negation words. Variant A, without stop-word removal, scored lowest (0.8793). With 1,500 validation reviews, one standard error of an accuracy near 0.89 is about 0.8 points, and the whole spread between variants is 1.1 points (about 16 reviews), so the ablation suggests that stop-word removal helps slightly but does not show that removing negation is clearly better than keeping it. Variant B was chosen for all remaining experiments because it had the best validation score.

TABLE I
PREPROCESSING ABLATION (VALIDATION MACRO-F1)

Variant	Macro-F1
A: no stop-word removal, lemma	0.8793
B: stop words removed incl. negation, lemma	**0.8899**
C: stop words removed, negation kept, lemma	0.8866
D: stop words removed, negation kept, stem	0.8886
E: stop words removed, negation kept, none	0.8873

B. Model comparison

Table II shows all the models on the test set. The best model is **TF-IDF (1+2-gram) + LogReg** ($C = 10$) with a test macro-F1 of **0.8847**, which is 0.5514 above the majority baseline (0.3333). Adding bigrams to unigram TF-IDF improved the LR model from 0.8720 to 0.8847 (about 19 more correct reviews out of 1,500) and Naive Bayes from 0.8525 to 0.8707; the consistent direction across both classifiers makes this gain credible, although we did not run a significance test. Logistic Regression beat Naive Bayes for every representation. Averaged Word2Vec was the weakest (0.8260), likely because averaging destroys word order and dilutes the rare but highly informative sentiment words, and because our embeddings were trained on only 7,000 reviews.

C. Confusion matrix

The best model makes 173 errors on the 1,500 test reviews (86 false positives and 87 false negatives, Fig. 6), meaning the model is not biased towards predicting one class.

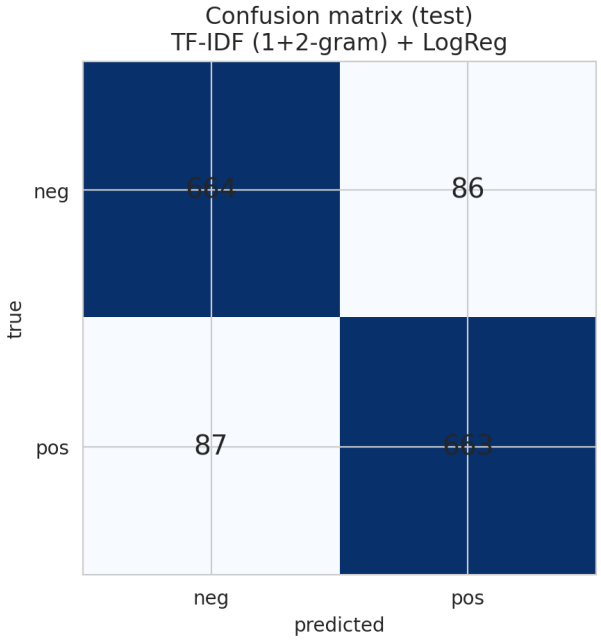


Fig. 6. Confusion matrix of the best model on the test set.

VII. ERROR ANALYSIS AND DISCUSSION

Informative words. By the Naive Bayes log-probability ratio (BoW model), the ten most informative positive words are *custer, jed, miyazaki, rififi, felix, clutter, rukh, fido, europa, gripping*, and the ten most informative negative words are *wasting, worst, waste, poorly, rubbish, awful, uwe, pointless, boll, ariel*. Although we expect both "worst" and "gripping" to be genuine sentiment words, the fact that we have particular names and titles (*custer, miyazaki, ariel*) indicates that the bag-of-words model can latch onto any entities that occur almost exclusively in one class; this has nothing to do with how well they express sentiment, and these features may not generalise.

Misclassification errors. We manually examined 20 incorrectly classified test reviews (10 false positives and 10 false negatives, drawn with a fixed seed) and grouped them into error types:

- **Negation (14):** the model fails when an incorrect sentence structure is used; if there are many words expressing very high levels of sentiment in a negative context, the model will fail. An example is "I would not go to that country no matter what."
- **Sarcasm (3):** this type occurs when large amounts of positive vocabulary are used to express a negative. Example: "Oh great, another two hours I'll never get back."
- **Mixed opinions (2):** these reviews contain both positive and negative views, for example praising the acting while criticising the script; it is impossible for the model to determine which view prevails.
- **Label noise / subtle cases (1):** in one case the model classified a positive cartoon review as negative, likely due to minor features or possibly a mislabelled data entry.

VIII. LIMITATIONS AND FUTURE WORK

We used a sample of 10,000 reviews instead of all 49,582 unique reviews, so we expect slight variations in results on the full dataset. All hyperparameters were tuned on one validation split with small grids, and several best values lie on a grid edge (BoW + LogReg $C = 0.1$, BoW + NB $\alpha = 3$, Word2Vec + LogReg $C = 100$); wider grids might change those models slightly. Differences of about one point, including the ablation, are within sampling noise, and we report no confidence intervals or significance tests. The error taxonomy comes from one 20-review sample read by the authors. Bag-of-words and averaged-embedding representations do not take into account the ordering of words within an utterance, thus they continue to have difficulty capturing sarcasm and individuals expressing both positive and negative opinions at the same time. Word2Vec was trained from a smaller vocabulary than would be needed to obtain a similar vector representation using a large dictionary such as GloVe. In the final-term project, we will train a pre-trained transformer (such as BERT) on the entire dataset and evaluate its performance against the previously evaluated traditional methods, and we also intend to determine whether training a pre-trained transformer on our dataset addresses some or all of the negation and sarcasm issues identified in this study.

TABLE II
TEST RESULTS FOR EVERY MODEL (HYPERPARAMETERS CHOSEN ON VALIDATION)

Representation + classifier	Best param.	Accuracy	Macro-P	Macro-R	Macro-F1
Majority class	–	0.5000	0.2500	0.5000	0.3333
BoW + MultinomialNB	$\alpha = 3$	0.8473	0.8491	0.8473	0.8471
BoW + LogReg	$C = 0.1$	0.8687	0.8688	0.8687	0.8687
TF-IDF (1-gram) + MultinomialNB	$\alpha = 1$	0.8527	0.8541	0.8527	0.8525
TF-IDF (1-gram) + LogReg	$C = 10$	0.8720	0.8722	0.8720	0.8720
TF-IDF (1+2-gram) + MultinomialNB	$\alpha = 1$	0.8707	0.8708	0.8707	0.8707
TF-IDF (1+2-gram) + LogReg	$C = 10$	**0.8847**	**0.8847**	**0.8847**	**0.8847**
Word2Vec avg + LogReg	$C = 100$	0.8260	0.8260	0.8260	0.8260

IX. CONCLUSION

We ran classical pipeline methods for IMDB sentiment classification using a standard train/validation/test approach. Our top method was TF-IDF with both 1-gram and 2-gram features and Logistic Regression on those features, which reached a test macro-F1 score of 0.8847, almost three times better than our baseline of 0.3333. The ablation experiment showed that removing stop-words from the vocabulary resulted in improved results. Additionally, we were able to run an error analysis that explains why bag-of-words models are still unable to adequately capture negative phrases, sarcasm and ambiguous statements.

REFERENCES

[1] B. Pang, L. Lee, and S. Vaithyanathan, "Thumbs up? Sentiment Classification Using Machine Learning Techniques," in *Proc. ACL-02 Conf. Empirical Methods in Natural Language Processing (EMNLP '02)*, vol. 10, 2002, pp. 79–86, doi: 10.3115/1118693.1118704.

[2] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning Word Vectors for Sentiment Analysis," in *Proc. 49th Annu. Meeting Assoc. Computational Linguistics: Human Language Technologies (ACL-HLT 2011)*, Portland, OR, USA, 2011, pp. 142–150. [Online]. Available: https://aclanthology.org/P11-1015/

[3] S. Wang and C. D. Manning, "Baselines and Bigrams: Simple, Good Sentiment and Topic Classification," in *Proc. 50th Annu. Meeting Assoc. Computational Linguistics (ACL '12)*, vol. 2, Jeju Island, Korea, 2012, pp. 90–94. [Online]. Available: https://aclanthology.org/P12-2018/

[4] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," *arXiv preprint arXiv:1301.3781*, 2013, doi: 10.48550/arXiv.1301.3781.

[5] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html

[6] S. M. Auti, "CSC4233 NLP Mid-Term Project - Group G07," Google Colab. [Online]. Available: https://colab.research.google.com/drive/1iOPpEYi0R4LVifP1c5mbay-Lx1qbWADt?usp=sharing

APPENDIX A
CONTRIBUTION STATEMENT

Both members contributed equally to understanding all parts of the code, worked hands-on with the models and tuning, and jointly prepared for the viva.

Signed off by:

Sadiya Mohon Auti (Auti) Tasnim Tabassum Mim (Mim)

TABLE III
CONTRIBUTION STATEMENT

Member	Tasks	Contr.
Sadiya Mohon Auti (23-52092-2)	Overall coordination and submission; preprocessing pipeline design and implementation; preprocessing ablation study; Naive Bayes and Logistic Regression training and hyperparameter tuning; notebook structure and code review; report writing (Sections I–VI, IX); final proofreading.	50%
Tasnim Tabassum Mim (23-50971-1)	Dataset loading, duplicate removal, sampling and splitting; EDA (Figures 1–4); BoW, TF-IDF and Word2Vec representation code; error analysis (top words and 20 misclassified reviews); notebook documentation; report writing (Sections VII–VIII).	50%

APPENDIX B
AI-USE STATEMENT

We used Claude (Anthropic) for learning and explanations of NLP topics, drafting the project proposal, structuring and drafting the notebook code and its explanatory text, structuring the report, checking reference information, and reviewing the draft report and notebook against the course guideline. All numbers in this report were produced by running the submitted notebook ourselves. We have read and understand all code and text, and our interpretations of the EDA, errors and conclusions are our own. The code follows the documented usage of scikit-learn, NLTK and gensim.